

Sujet de TP

Construire un RAG

Films - Medicaments - Code du Travail - Au choix

Niveau : Intermediaire

Duree estimee : 3 a 4 heures

Outils : Groq, FAISS, sentence-transformers

TP - Construire un RAG de bout en bout avec Python et Groq

Niveau : Intermédiaire

Durée estimée : 3 à 4 heures

Prérequis : Bases de Python, notions de POO, avoir suivi le cours sur les RAG

Sommaire

1. Présentation du TP
 2. Mise en place de l'environnement
 3. Choix du sujet
 - Sujet A - Recommandation de Films
 - Sujet B - Assistant Médicaments
 - Sujet C - Assistant Code du Travail
 4. Partie commune - Construction du RAG
 5. Critères d'évaluation
-

1. Présentation du TP

Objectif

Dans ce TP, vous allez construire un système **RAG (Retrieval-Augmented Generation) complet**, de l'ingestion des données jusqu'à l'interface de questions-réponses. Vous choisirez librement votre sujet parmi les trois proposés.

À la fin du TP, votre système devra être capable de :

- Charger et préparer une base de connaissance réelle
- Découper les documents en chunks cohérents
- Créer et persister une base de données vectorielle (FAISS)
- Répondre à des questions en langage naturel en citant ses sources
- Refuser de répondre si l'information n'est pas dans sa base de connaissance

Architecture cible

■ PHASE 1 : INDEXATION ■

■ (à exécuter une fois) ■

■ ■

■ Données brutes → Nettoyage → Chunking → Embedding ■

■ ↓ ■

■ Base FAISS ■

↓

■ PHASE 2 : INTERROGATION ■

■ (à chaque question) ■

■ ■

■ Question → Embedding → Recherche vectorielle ■

■ ↓ ■

■ Top-k chunks ■

■ ↓ ■

■ LLM Groq + Contexte ■

■ ↓ ■

■ Réponse avec sources ■

Ce qui est attendu

Vous rendrez un dossier contenant :

- `indexation.py` - le script de création de la base vectorielle
- `rag.py` - le système de questions-réponses
- `README.md` - expliquant vos choix techniques et comment lancer le projet
- Un compte-rendu court (1 page) décrivant les difficultés rencontrées et vos décisions de conception

Important : Vous n'avez pas le droit d'utiliser LangChain ou LlamaIndex pour ce TP. L'objectif est de comprendre chaque brique en l'implémentant vous-même.

2. Mise en place de l'environnement

Création d'un environnement virtuel

```
python -m venv venv

source venv/bin/activate # Linux / Mac

venv\Scripts\activate # Windows
```

Installation des dépendances

```
pip install groq

pip install sentence-transformers

pip install faiss-cpu

pip install requests

pip install python-dotenv
```

Créez un fichier `requirements.txt` listant toutes vos dépendances. C'est une bonne pratique de développement.

Configuration de la clé API Groq

1. Créez un compte gratuit sur console.groq.com
2. Générez une clé API dans la section **API Keys**
3. Créez un fichier `.env` à la racine de votre projet :

```
GROQ_API_KEY=votre_clé_ici
```

4. Chargez-la dans votre code :

```
from dotenv import load_dotenv

import os

load_dotenv()

api_key = os.getenv("GROQ_API_KEY")
```

Ne commitez jamais votre clé API dans Git. Ajoutez `.env` à votre `.gitignore`.

Vérification de l'installation

Avant de commencer, vérifiez que tout fonctionne avec ce script de test :

```
from groq import Groq

from sentence_transformers import SentenceTransformer

import faiss

import numpy as np


# Test Groq

client = Groq(api_key="votre_clé")

response = client.chat.completions.create(

    model="llama3-8b-8192",

    messages=[{"role": "user", "content": "Dis bonjour en une phrase."}]

)

print("■ Groq OK :", response.choices[0].message.content)


# Test Embeddings

model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")

vector = model.encode("Test d'embedding")

print(f"■ Embedding OK – dimension : {len(vector)}")


# Test FAISS

index = faiss.IndexFlatL2(768)

index.add(np.array([vector], dtype=np.float32))

print(f"■ FAISS OK – {index.ntotal} vecteur(s) indexé(s)")
```

3. Choix du sujet

Lisez les trois sujets ci-dessous et choisissez-en **un seul**. Chaque sujet vous guide vers une source de données réelle et gratuite, et vous pose des questions de réflexion spécifiques. La partie 4 est ensuite commune à tous les sujets.

Sujet A - Recommandation de Films

Contexte

Vous allez construire un assistant de recommandation de films capable de répondre à des requêtes en langage naturel comme :

- **"Je cherche un thriller psychologique avec un retournement de situation inattendu"**
- **"Recommande-moi un film d'animation familial sorti après 2010"**
- **"Un film comme Inception mais plus accessible ?"**

La particularité de ce sujet est que vous travaillez avec des **données tabulaires** (un CSV), que vous devrez convertir en texte avant de les indexer.

Accès aux données

Dataset utilisé : TMDb 5000 Movie Dataset

Ce dataset est librement disponible sur Kaggle. Il contient environ 5 000 films avec leurs métadonnées complètes.

Étapes pour télécharger les données

1. Créez un compte gratuit sur [kaggle.com](https://www.kaggle.com)
2. Rendez-vous sur la page du dataset : [kaggle.com/datasets/tmdb/tmdb-movie-metadata](https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata)
3. Cliquez sur **Download** et téléchargez le fichier `tmdb_5000_movies.csv`
4. Placez le fichier dans un dossier `data/` à la racine de votre projet

Structure du CSV

Le fichier contient les colonnes suivantes (entre autres) :

Colonne	Description
<code>title</code>	Titre du film
<code>overview</code>	Synopsis en anglais
<code>genres</code>	Genres au format JSON
<code>release_date</code>	Date de sortie
<code>vote_average</code>	Note moyenne (sur 10)
<code>vote_count</code>	Nombre de votes
<code>runtime</code>	Durée en minutes
<code>original_language</code>	Langue originale

Chargement des données

```
import pandas as pd

import json

df = pd.read_csv("data/tmdb_5000_movies.csv")

print(f"Nombre de films : {len(df)}")

print(df.columns.tolist())

print(df.head(2))
```

Questions de réflexion préliminaires

Avant d'écrire votre code, réfléchissez à ces questions et notez vos réponses dans votre README :

- Q1.** Les données sont tabulaires (CSV), pas du texte libre. Comment allez-vous convertir chaque ligne en un texte cohérent et riche en information à embedder ? Quelles colonnes inclure en priorité ?
- Q2.** La colonne genres est au format JSON imbriqué (ex: [{"id": 18, "name": "Drama"}]). Comment allez-vous l'extraire proprement ?
- Q3.** Avec 5 000 films, l'indexation peut prendre plusieurs minutes. Quelle stratégie adoptez-vous pour ne pas avoir à relancer l'indexation à chaque test ?
- Q4.** Une recommandation de film n'est pas une question factuelle (il n'y a pas de "bonne" réponse). Comment allez-vous guider le LLM dans le prompt système pour qu'il fasse des recommandations pertinentes et argumentées ?
- Q5.** Que faire si l'utilisateur demande un film très récent (2024) qui n'est pas dans votre base de données de 2017 ? Comment le système doit-il se comporter ?

Contraintes spécifiques

- Votre index FAISS devra contenir **au minimum 500 films**
- Pour chaque recommandation, le LLM devra citer le titre du film et sa note
- Vous implémenterez un **filtre par langue** : l'utilisateur pourra préciser s'il veut des films en VO française ou internationale

Exemple de sortie attendue

■ Question : Un film de science-fiction sur l'intelligence artificielle,
avec une réflexion philosophique profonde ?

■ Recommandations :

1. **Ex Machina** (2014) – Note : 7.7/10

Un film intimiste et glaçant sur la création d'une IA humanoïde.

Explore les thèmes de la conscience, de la manipulation et de ce qui définit l'humanité. Parfait si vous cherchez un thriller psychologique lent et réfléchi.

2. **Her** (2013) – Note : 8.0/10

[...]

■ Sources : films 1247, 832, 2901 de la base de données

Sujet B - Assistant Médicaments

Contexte

Vous allez construire un assistant d'information sur les médicaments qui répond à des questions comme :

- "Quels sont les effets secondaires du Doliprane ?"
- "Y a-t-il des contre-indications entre l'ibuprofène et l'aspirine ?"
- "Quelle est la posologie recommandée pour l'amoxicilline adulte ?"

Avertissement éthique important : Votre système devra systématiquement rappeler qu'il ne remplace pas un avis médical et encourager la consultation d'un professionnel de santé. C'est une contrainte technique du TP, pas seulement éthique.

Accès aux données

Les données proviennent de la **Base de Données Publique des Médicaments (BDPM)**, une source officielle open data gérée par l'ANSM, mise à disposition sur data.gouv.fr.

Vous avez deux options pour récupérer les données :

Option 1 - Téléchargement direct (recommandée pour débiter)

Rendez-vous sur : data.gouv.fr/datasets/base-de-donnees-publique-des-medicaments-base-officielle

Téléchargez le fichier `CIS_RCP.zip` (Résumés des Caractéristiques des Produits).

Ce fichier contient les notices complètes de chaque médicament : indications thérapeutiques, posologie, contre-indications, effets indésirables, etc. C'est exactement le contenu dont vous avez besoin pour un RAG.

Les fichiers sont encodés en **Latin-1 (ISO-8859-1)**. Pensez à le préciser lors de la lecture avec `pandas` ou `open()`.

```
import zipfile

import pandas as pd

# Décompresser le fichier

with zipfile.ZipFile("data/CIS_RCP.zip", "r") as z:

    z.extractall("data/")

# Lire le CSV (attention à l'encodage)

df = pd.read_csv("data/CIS_RCP.txt", sep="\t", encoding="latin-1", header=None)

print(df.head())
```

Option 2 - Via l'API REST (pour aller plus loin)

Une API publique et **sans clé d'authentification** permet d'interroger la BDPM directement :

```
Base URL : https://api.medicaments.gouv.fr/1.0
```

Exemple de requête pour récupérer les informations d'un médicament par son nom :

```
import requests

def rechercher_medicament(nom: str) -> dict:

    url = f"https://api.medicaments.gouv.fr/1.0/medicaments"

    params = {"denomination": nom}

    response = requests.get(url, params=params)

    return response.json()

# Exemple

data = rechercher_medicament("doliprane")

print(data)
```

L'API retourne des données structurées en JSON. Vous devrez les convertir en texte avant de les indexer, comme pour le sujet A.

Constitution du corpus pour le TP

Pour ce TP, vous travaillerez sur un **corpus limité de 15 à 20 médicaments courants**. Cela suffit à démontrer le RAG et évite des temps d'indexation trop longs.

Voici une liste de départ suggérée :

```
medicaments_corpus = [

    "doliprane", "dafalgan", "efferalgan", # paracétamol

    "ibuprofène", "advil", "nurofen", # ibuprofène

    "aspirin", "aspégic", # aspirine

    "amoxicilline", "augmentin", # antibiotiques

    "smecta", "imodium", # digestif

    "ventoline", "becotide", # asthme

    "oméprazole", "inexium", # estomac

    "metformine", "glucophage" # diabète

]
```

Questions de réflexion préliminaires

Q1. Les notices de médicaments sont des documents très longs et très denses (plusieurs pages). Quelle stratégie de chunking adoptez-vous ? Quelle taille de chunk vous semble appropriée pour ce type de contenu ?

Q2. Une notice est structurée en sections bien définies (indications, posologie, contre-indications, effets indésirables, interactions médicamenteuses...). Pouvez-vous exploiter cette structure pour améliorer votre chunking ?

Q3. Comment distinguer dans votre base vectorielle les chunks relatifs aux effets secondaires de ceux relatifs à la posologie ? Les métadonnées peuvent-elles aider ?

Q4. Un utilisateur demande : `"Puis-je prendre du Doliprane et de l'ibuprofène en même temps ?"`. Cette question nécessite des informations sur **deux** médicaments. Comment votre système peut-il gérer ce cas ?

Q5. Comment formuler le prompt système pour que le LLM soit à la fois informatif et systématiquement prudent sur les recommandations médicales ?

Contraintes spécifiques

- Chaque réponse devra **obligatoirement** inclure la mention : `"Ces informations ne remplacent pas l'avis d'un professionnel de santé."`
- Le LLM devra indiquer clairement **de quel médicament** provient chaque information
- Si la question dépasse le périmètre des médicaments indexés, le système doit le signaler explicitement plutôt qu'inventer une réponse

Exemple de sortie attendue

■ Question : Quels sont les effets secondaires courants de l'ibuprofène ?

■ Réponse :

D'après la notice officielle de l'ibuprofène, les effets indésirables les plus fréquents sont :

- Troubles digestifs : nausées, vomissements, douleurs abdominales, diarrhées (fréquents, surtout si pris sans nourriture)
- Maux de tête et vertiges (peu fréquents)
- Réactions allergiques cutanées (rares)
- En cas d'utilisation prolongée : risque d'ulcère gastrique

■ Ces informations ne remplacent pas l'avis d'un professionnel de santé.

En cas de doute, consultez votre médecin ou votre pharmacien.

■ Source : Notice officielle ibuprofène – section "Effets indésirables"

Sujet C - Assistant Code du Travail

Contexte

Vous allez construire un assistant juridique capable de répondre à des questions sur le droit du travail français :

- `""Quelle est la durée légale du préavis pour un CDI ?""`
- `""Combien d'heures supplémentaires peut-on faire par semaine ?""`
- `""Quels sont mes droits en cas de licenciement économique ?""`
- `""Comment fonctionne la rupture conventionnelle ?""`

Avertissement juridique : Comme pour le sujet médical, votre système devra systématiquement rappeler qu'il ne remplace pas l'avis d'un avocat ou d'un juriste.

Accès aux données

Le Code du travail français est un **document public et librement réutilisable**. Il est accessible via plusieurs sources.

Option 1 - API Légifrance (recommandée)

Légifrance propose une **API officielle et gratuite** pour accéder aux textes législatifs.

Inscription :

1. Rendez-vous sur api.gouv.fr/les-api/api-legifrance
2. Cliquez sur "**Demander l'accès**"
3. Remplissez le formulaire (accès accordé rapidement pour usage pédagogique)
4. Vous recevrez un `client_id` et un `client_secret`

Authentification (OAuth2) :

```
import requests

def get_legifrance_token(client_id: str, client_secret: str) -> str:

    """Obtient un token d'accès OAuth2 pour l'API Légifrance."""

    url = "https://oauth.piste.gouv.fr/api/oauth/token"

    response = requests.post(url, data={

        "grant_type": "client_credentials",

        "client_id": client_id,

        "client_secret": client_secret,

        "scope": "openid"

    })

    return response.json()["access_token"]
```

Récupération des articles :

```
def get_article(token: str, cid_article: str) -> dict:

    """Récupère le texte d'un article du Code du travail."""

    url = "https://api.piste.gouv.fr/dila/legifrance/lf-engine-app/consult/getArticle"

    headers = {"Authorization": f"Bearer {token}"}

    payload = {"id": cid_article}

    response = requests.post(url, json=payload, headers=headers)

    return response.json()
```

Option 2 - Téléchargement depuis data.gouv.fr (plus simple)

Le Code du travail est disponible en téléchargement direct au format XML ou JSON sur :

data.gouv.fr/datasets/legi-data

Téléchargez le fichier **LEGI** correspondant au Code du travail. Il est au format XML et contient l'intégralité des articles.

```
import xml.etree.ElementTree as ET

def extraire_articles_code_travail(fichier_xml: str) -> list[dict]:

    """Extrait les articles du Code du travail depuis le XML LEGI."""

    tree = ET.parse(fichier_xml)

    root = tree.getroot()

    articles = []

    # À vous de parcourir l'arbre XML et d'extraire

    # les balises <ARTICLE> avec leur numéro et leur texte

    # ...

    return articles
```

Option 3 - Corpus manuel (si les deux options précédentes posent problème)

Si vous rencontrez des difficultés avec les API, vous pouvez constituer un corpus **manuellement** en copiant-collant les articles depuis legifrance.gouv.fr.

Concentrez-vous sur les thèmes les plus consultés :

```

themes_prioritaires = [

    "Durée du travail et heures supplémentaires", # L3121-1 à L3121-36

    "Congés payés", # L3141-1 à L3141-32

    "Contrat de travail (CDI, CDD)", # L1221-1 à L1248-11

    "Licenciement", # L1231-1 à L1237-20

    "Rupture conventionnelle", # L1237-11 à L1237-19

    "Salaire minimum (SMIC)", # L3231-1 à L3232-9

    "Représentation du personnel", # L2311-1 à L2316-26

    "Harcèlement et discrimination", # L1152-1 à L1155-2

]

```

Créez un fichier corpus/code_travail.json structuré ainsi :

```

[

  {

    "article": "L3121-1",

    "titre": "Durée légale du travail",

    "texte": "La durée légale du travail effectif des salariés...",

    "section": "Durée du travail"

  }

]

```

Questions de réflexion préliminaires

Q1. Les articles de loi sont courts mais très denses juridiquement, et ils se réfèrent souvent les uns aux autres ("au sens de l'article L1234-5..."). Faut-il indexer chaque article séparément ou les regrouper par section ? Quels sont les avantages et inconvénients de chaque approche ?

Q2. Le numéro d'article (ex: L3121-1) est une information cruciale pour la traçabilité. Comment l'intégrez-vous dans vos métadonnées ? Comment le LLM doit-il le citer dans sa réponse ?

Q3. Le droit du travail évolue régulièrement (lois, ordonnances). Comment votre système peut-il indiquer à l'utilisateur que l'information peut être obsolète ?

Q4. Un utilisateur peut poser une question dont la réponse dépend de **son secteur d'activité** ou de la **taille de l'entreprise** (ex: les seuils varient selon que l'entreprise a +50 salariés). Comment gérer ces cas dans votre prompt et votre réponse ?

Q5. Comment distinguer dans votre RAG une question à laquelle le Code du travail répond directement d'une question qui nécessite une interprétation juridique (auquel cas le système doit orienter vers un juriste) ?

Contraintes spécifiques

- Votre corpus devra couvrir **au minimum 5 thèmes différents** du Code du travail
- Chaque réponse devra **citer le numéro d'article** sur lequel elle se base
- Le LLM devra être capable de dire **"Je ne trouve pas cette information dans ma base"** plutôt que d'inventer
- Chaque réponse inclura la mention : **"Cet assistant ne fournit pas de conseil juridique. Consultez un avocat ou l'inspection du travail pour votre situation."**

Exemple de sortie attendue

■ Question : Combien de jours de congés payés ai-je droit par an ?

■■ Réponse :

Selon l'article L3141-3 du Code du travail, tout salarié a droit à un congé de deux jours et demi ouvrables par mois de travail effectif chez le même employeur, soit 30 jours ouvrables (5 semaines) par an.

Précisions importantes :

- Le calcul se base sur les mois de travail effectif (article L3141-4)
- Des jours supplémentaires peuvent s'ajouter selon la convention collective applicable dans votre entreprise
- La période de référence est généralement du 1er juin au 31 mai

■ Sources : Articles L3141-3, L3141-4 du Code du travail

■■ Cet assistant ne fournit pas de conseil juridique. Consultez un avocat ou l'inspection du travail pour votre situation personnelle.

4. Partie commune - Construction du RAG

Une fois vos données récupérées et comprises, vous allez implémenter le pipeline RAG. Les étapes ci-dessous s'appliquent à tous les sujets.

Étape 1 - Préparation des données

Quelle que soit votre source, l'objectif est d'obtenir une liste de dictionnaires avec cette structure :

```
documents = [  
  
    {  
  
        "id": "doc_001",  
  
        "contenu": "Texte complet et nettoyé de ce document...",  
  
        "metadata": {  
  
            "source": "nom_du_fichier_ou_url",  
  
            # + champs spécifiques selon votre sujet  
  
            # Films : "titre", "annee", "note", "genres"  
  
            # Médics : "medicament", "section"  
  
            # Droit : "article", "titre_section"  
  
        }  
  
    },  
  
    # ...  
  
]
```

Questions à vous poser :

- Quels champs inclure dans le texte à embedder pour maximiser la pertinence de la recherche ?
 - Quels champs garder uniquement en métadonnées (non embeddés, mais récupérables) ?
 - Y a-t-il des données à nettoyer (balises HTML, caractères spéciaux, encodage...) ?
-

Étape 2 - Chunking

Implémentez une fonction `chunker(texte, taille_max, overlap)` qui découpe un texte long en morceaux cohérents.

```
def chunker(texte: str, taille_max: int = 500, overlap: int = 50) -> list[str]:  
  
    """  
  
    Découpe un texte en chunks avec chevauchement.  
  
    Args:  
  
        texte: le texte à découper  
  
        taille_max: nombre maximum de caractères par chunk  
  
        overlap: nombre de caractères répétés entre deux chunks consécutifs  
  
    Returns:  
  
        Liste de chunks  
  
    """  
  
    # À vous d'implémenter  
  
    pass
```

Questions à vous poser :

- Quelle taille de chunk est adaptée à votre contenu ? (les notices médicales ne se découpent pas comme des synopses de films)
- Quel séparateur prioritaire utiliser ? (`\n\n` pour les paragraphes ? Les numéros d'articles ?)
- Quel overlap est raisonnable pour votre cas ?

Testez votre chunker sur quelques exemples avant de l'appliquer à tout le corpus. Affichez les chunks obtenus et vérifiez qu'ils ont du sens.

Étape 3 - Création des embeddings

Choisissez un modèle d'embedding adapté à votre contenu.

```
from sentence_transformers import SentenceTransformer

# Pour du contenu en français ou multilingue

modele = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")

# Pour du contenu en anglais uniquement (sujet Films)

# modele = SentenceTransformer("all-mpnet-base-v2")

def embedder_chunks(chunks: list[str], modele) -> np.ndarray:

    """

    Transforme une liste de chunks en vecteurs.

    Returns:

        Tableau numpy de shape (n_chunks, dimension)

    """

    # À vous d'implémenter

    pass
```

Questions à vous poser :

- Votre contenu est-il principalement en français ou en anglais ? Quel modèle choisir en conséquence ?
- L'encodage de tout le corpus peut prendre du temps. Comment afficher une barre de progression ?
- Comment vérifier que les vecteurs obtenus ont la bonne dimension ?

Étape 4 - Création et persistance de la base FAISS

```
import faiss

import numpy as np

import json


def creer_index_faiss(vecteurs: np.ndarray) -> faiss.Index:
    """
    Crée un index FAISS à partir des vecteurs.

    Args:
        vecteurs: tableau numpy de shape (n, dimension)

    Returns:
        Index FAISS prêt à l'emploi
    """
    # À vous d'implémenter

    pass


def sauvegarder_index(index, chunks_avec_meta: list, chemin: str):
    """
    Sauvegarde l'index FAISS et les métadonnées associées sur disque.

    L'index FAISS associe chaque vecteur à un entier (son rang).
    Il faut donc aussi sauvegarder la liste des chunks dans le même ordre
    pour retrouver le texte correspondant à un résultat de recherche.
    """
    # À vous d'implémenter

    # Indice : faiss.write_index() pour l'index

    # json.dump() pour les métadonnées
```

```

pass

def charger_index(chemin: str):

    """Recharge l'index depuis le disque sans réindexer."""

    # À vous d'implémenter

    pass

```

Questions à vous poser :

- Pourquoi est-il important de sauvegarder les chunks **dans le même ordre** que les vecteurs FAISS ?
- Quelle est la différence entre IndexFlatL2 et IndexFlatIP dans FAISS ? Lequel utiliser pour la similarité cosinus ?
- Comment vérifiez-vous que votre index a bien été sauvegardé et rechargé correctement ?

Étape 5 - Fonction de recherche

```

def rechercher(question: str, modele, index, chunks_avec_meta: list, k: int = 4) -> list[dict]:

    """

    Recherche les k chunks les plus pertinents pour une question.

    Args:

        question: la question de l'utilisateur

        modele: le modèle d'embedding (même que celui utilisé à l'indexation)

        index: l'index FAISS

        chunks_avec_meta: la liste des chunks avec leurs métadonnées

        k: nombre de résultats à retourner

    Returns:

        Liste de dicts {"contenu": ..., "metadata": ..., "score": ...}

    """

    # À vous d'implémenter

    pass

```

Questions à vous poser :

- Que représente le score retourné par FAISS ? Un score élevé est-il meilleur ou moins bon ?
- Faut-il normaliser les vecteurs avant la recherche ? Pourquoi ?
- Comment tester que votre fonction de recherche retourne des résultats pertinents **avant** de brancher le LLM ?

Test conseillé : Avant de passer à l'étape suivante, testez votre fonction de recherche sur 5 questions et affichez les chunks retournés. Si les résultats semblent aberrants, votre problème vient du chunking ou de l'embedding, pas du LLM.

Étape 6 - Génération de la réponse avec Groq

```
from groq import Groq

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

def construire_prompt_systeme() -> str:
    """
    Retourne le prompt système adapté à votre sujet.

    Ce prompt définit le comportement général du LLM.

    Réfléchissez à :

    - Le rôle que joue le LLM (expert films ? assistant médical ?)

    - Les contraintes (ne pas inventer, citer les sources, avertissements)

    - Le format de la réponse attendue

    """
    # À vous d'implémenter

    pass

def generer_reponse(question: str, chunks_pertinents: list[dict]) -> str:
    """
    Génère une réponse en utilisant les chunks comme contexte.

    Args:

        question: la question de l'utilisateur

        chunks_pertinents: les résultats de la recherche vectorielle

    Returns:

        La réponse générée par le LLM

    """
```



```
# Assemblez le contexte à partir des chunks

# Appelez l'API Groq avec le bon prompt

# Retournez la réponse

# À vous d'implémenter

pass
```

Questions à vous poser :

- Comment présenter les chunks au LLM dans le prompt ? Faut-il les numéroté ? Inclure les métadonnées ?
- Que se passe-t-il si vous envoyez trop de chunks ? Le contexte a une taille maximale.
- Comment forcer le LLM à ne répondre que sur la base du contexte fourni ?
- Quel modèle Groq choisir ? llama3-8b-8192 (rapide) ou llama3-70b-8192 (meilleur, mais plus lent) ?

Étape 7 - Interface en ligne de commande

Assemblez toutes les briques dans un script `rag.py` avec une boucle interactive :

```
def main():

    print("Chargement de la base de connaissances...")

    # Charger l'index et les métadonnées

    print("■ Système RAG prêt. Tapez 'quit' pour quitter.\n")

    while True:

        question = input("■ Votre question : ").strip()

        if question.lower() in ["quit", "exit", "q"]:

            print("Au revoir !")

            break

        if not question:

            continue

        # 1. Rechercher les chunks pertinents

        # 2. Générer la réponse

        # 3. Afficher la réponse et les sources

if __name__ == "__main__":

    main()
```

Étape 8 (Bonus) - Amélioration du pipeline

Si vous avez terminé les étapes précédentes, implémentez **au moins une** des améliorations suivantes :

Bonus A - Gestion de l'historique

Permettez au LLM de tenir compte des échanges précédents dans la conversation (pas seulement la dernière question).

Bonus B - Score de confiance

Affichez le score de similarité des chunks retournés. Si le meilleur score est inférieur à un seuil, affichez un avertissement `""Je n'ai pas trouvé d'information directement pertinente dans ma base.""`

Bonus C - Reformulation de la question

Avant la recherche vectorielle, demandez au LLM de reformuler la question de l'utilisateur pour la rendre plus pertinente à la recherche (ex: transformer `""j'ai mal à la tête, que prendre ?""` en `""médicaments antidouleur céphalées posologie""`).

Bonus D - Mode comparaison

Permettez à l'utilisateur de comparer deux éléments (deux films, deux médicaments, deux articles de loi) en récupérant les chunks des deux et en demandant au LLM de faire une synthèse comparative.

5. Critères d'évaluation

Critère	Points	Détail
Fonctionnement du pipeline complet	5 pts	Indexation + recherche + génération fonctionnels
Qualité du chunking	3 pts	Chunks cohérents, overlap adapté, taille justifiée
Qualité du prompt	3 pts	Comportement du LLM maîtrisé, sources citées, avertissements présents
Persistence de la base	2 pts	L'index FAISS est sauvegardé et rechargeable sans réindexation
Qualité du code	3 pts	Lisible, commenté, fonctions bien découpées
README et compte-rendu	2 pts	Choix techniques justifiés, difficultés documentées
Bonus	2 pts	Au moins un bonus implémenté et fonctionnel
Total	20 pts	

Erreurs éliminatoires

- Utilisation de LangChain ou LlamaIndex
- Clé API committée dans le code source
- Code entièrement généré par IA sans compréhension démontrée (questions orales)
- Absence de persistance (l'index est recréé à chaque lancement)

Annexes

Commandes utiles

```
# Vérifier la version des packages installés

pip list | grep -E "groq|faiss|sentence"


# Générer le requirements.txt

pip freeze > requirements.txt


# Tester l'API Groq en ligne de commande (curl)

curl https://api.groq.com/openai/v1/chat/completions \

  -H "Authorization: Bearer $GROQ_API_KEY" \

  -H "Content-Type: application/json" \

  -d '{"model": "llama3-8b-8192", "messages": [{"role": "user", "content": "Bonjour"}]}'
```

Ressources

- Documentation Groq : console.groq.com/docs
- Documentation FAISS : github.com/facebookresearch/faiss/wiki
- Documentation sentence-transformers : sbert.net
- BDPM (données médicaments) : data.gouv.fr
- API Légifrance : api.gouv.fr/les-api/api-legifrance
- Dataset films TMDB : kaggle.com/datasets/tmdb/tmdb-movie-metadata

Document pédagogique - Cours RAG & LLM